ADVANCE WEB
CLASS 2

Instructor: Dr. Mazin Md. Alkathiri
Deputy Dean College of Computers
Head of Information Technology Department
Seiyun University – Yemen
Sept. 2024

The Philosophy of Modularity: Projects vs. Apps

Django is designed for "Rapid Development" and "Clean Design". To achieve this, it separates a website into two distinct layers:

- **A. The Django Project**
  - **Definition:** The entire web application system. It is the "container" for your website.
  - **Role:** It manages global configurations, such as database connections, security settings, and the main URL entry point.
  - **Analogy:** The **Project** is like a shopping mall. It provides the building, electricity, and security, but it doesn't sell products itself.
- **B. The Django App**
  - **Definition:** A self-contained module that performs one specific job.
  - **Role:** Each app handles a single feature. For a "School Portal," you might have a students app, an accounts app for login, and a courses app.
  - **Benefit:** Apps are "pluggable." You can copy an app from one project and drop it into another with very little work.

3.1 Project vs App (Very Important)

✅ **Django Project:** A **project** is the whole website (settings, main URLs, configurations).

- Example: school_project

✅ **Django App:** An **app** is a part of the website that does one job.

- Examples of apps:
  - students (student pages)
  - accounts (login/register)
  - courses (course pages)
- 📌 A project can contain **many apps**.

Section 3: Creating Django Apps + First View + URL Routing

In this section, we will build a **mini project** called **School Portal**.
You will learn how to:

- ✅ Create a Django **app**
- ✅ Write your **first view**
- ✅ Connect it using **URL routing**
- ✅ Create a simple **HTML template page**
- ✅ (Bonus) Use a dynamic URL like /student/5/

3.2 Mini Project Setup: “School Portal”

Step 1: Go to your project folder

- Example:
- cd C:\django_course\mysite
- (If you’re starting fresh, make sure your venv is activated.)

Step 2: Confirm server works

- python manage.py runserver
- Stop it with **CTRL + C** before continuing.

3.3 Creating a Django App

- We will create an app called: **students**
- Run: `python manage.py startapp students`
  - Now your project will look like:

```
mysite/
│
├── manage.py
├── mysite/
│   ├── settings.py
│   ├── urls.py
│   └── ...
└── students/
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    └── views.py
```

- This generates a folder with specialized files:
  - views.py: Where the logic for your pages lives.
  - models.py: Where you define your database tables.
  - admin.py: Where you register your data for the Admin Dashboard.

3.4 Register the App in settings.py

Django is secure and ignores any code that is not explicitly registered. You must tell the **Project** that the **App** exists.

- Open: mysite/settings.py
- Find INSTALLED_APPS then add the app name 'students', to the INSTALLED_APPS list.:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'students',  # ✅ add this line
]
```

- ✅ Now Django knows your app exists.

3.5 The Logic Layer: Understanding Views

The **View** is the "Brain" of your application. It acts as the bridge between the user's request and the final data displayed on the screen.

- The Request-Response Cycle
  - **Request:** The user enters a URL in their browser.
  - **Logic:** Django finds the correct view function and passes it an HttpRequest object.
  - **Action:** The view interacts with models (data) or performs calculations.
  - **Response:** The view returns an HttpResponse (text) or a rendered Template (HTML).

Your First View (Hello Page)

- Open: students/views.py
- Replace the file content with:

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Welcome to the School Portal!")
```

What is a View?

A **view** is a Python function that:

- receives an HTTP request
- returns an HTTP response (text, HTML, JSON, etc.)

3.6 The Mapping Layer: URL Routing

If the View is the "Brain," the **URL Configuration** is the "Map". We use a two-tier routing system to keep code organized.

Tier 1: App-Level Routing

- Create a urls.py file *inside* the students folder.

Tier 2: Project-Level Routing

- Connect the app's URLs to the main project using the include() function.
  - **Action:** Open mysite/urls.py.
  - **Code:** path('students/', include('students.urls')),.
  - **Result:** All student-related pages will now start with /students/.

URL Routing (Connecting View to a URL)

- We need to connect:
  - URL like /students/

  to
  - view function home

Step 1: Create students/urls.py

- Inside the students folder, create a new file: ✅ students/urls.py
  - Add this code:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='students_home'), # Empty path means 'home'
]
```

- This means:
  - when someone visits /students/
  - Django will run home()

Connect App URLs to Project URLs

- Open: mysite/urls.py
- Update it like this:

```
from django.contrib import admin
from django.urls import path, include   # ✅ include added

urlpatterns = [
    path('admin/', admin.site.urls),
    path('students/', include('students.urls')),  # ✅ connect app URLs

]
```

3.8 Run and Test

- Run: python manage.py runserver
- Open your browser:
- ✅ Admin page:
  - http://127.0.0.1:8000/admin/
- ✅ Students home page:
  - http://127.0.0.1:8000/students/
- You should see:
  - Welcome to the School Portal!

Mini Project Upgrade: Use Templates (Real Web Page)

Right now we return plain text. Let’s return a real HTML page.

The Presentation Layer: Templates

Returning plain text is not professional. We use **Templates** to return real HTML.

- **Theory - Separation of Concerns:** Programmers write Python in views.py; designers write HTML in home.html.
- **Structure:** Django looks for templates in a specific folder path: students/templates/students/home.html.
- **The Render Function:** We use render(request, 'students/home.html') to combine the logic with the design.

A. Theory: The "Separation of Concerns" (SoC)

The primary reason we use templates is to separate the **Business Logic** from the **User Interface (UI)**.

- **Logic (Python):** Handled in views.py. This includes fetching data from a database or performing calculations.
- **Presentation (HTML):** Handled in .html files. This focuses on how the information looks to the user.
- **The Benefit:** A web designer can edit the HTML/CSS without ever touching (or accidentally breaking) the Python code.

B. The Structure: The "Namespacing" Rule

Django is built to handle many apps at once. If every app has an index.html, Django might get confused about which one to show. To prevent this, we use **Namespacing**.

- **The Standard Directory Path:**
- students/templates/students/home.html
- **students/**: The app folder.
- **templates/**: The folder Django is programmed to look for.
- **students/**: A sub-folder that matches the app name (the "Namespace").
- **home.html**: The actual file.

C. The Render Function: "Baking" the Page

We transition from HttpResponse to the render() shortcut. Think of render() as a chef combining ingredients.

- **The Syntax:** `return render(request, 'students/home.html', context)`

  - **request**: The initial user request (passed so Django knows who is asking).
  - **template_name**: The path to your HTML file starting after the templates/ folder.
  - **context (Optional)**: A dictionary of Python data you want to display on the page.

D. Introduction to Django Template Language (DTL)

Templates are not just static HTML; they are **dynamic**. DTL allows us to write logic inside HTML using special tags:

- **Variables** {{ }}: Used to print data from the view.
  - *Example:* <h1>Hello {{ student_name }}</h1>
- **Tags** {% %}: Used for logic like loops or "If" statements.
  - *Example:* {% if student_passed %} Congrats! {% endif %}

E. Theory: Security in Templates

- Using Django templates provides automatic protection against **Cross-Site Scripting (XSS)**.

- Django "escapes" HTML by default, meaning if a user tries to type <script>bad_code</script> into a form, Django renders it as harmless text rather than executing it as code.

1. What is XSS? (The Threat)

Cross-Site Scripting (XSS) occurs when an attacker injects malicious scripts (usually JavaScript) into a webpage viewed by other users.

- **The Scenario:** Imagine a "Student Comments" section in your School Portal.
- **The Attack:** An attacker submits a comment containing: <script>alert('Your account is hacked!');</script>.
- **The Risk:** Without protection, every student who visits that page would execute that script in their browser, potentially allowing the attacker to steal login cookies or session data.

2. How Django Protects You: Automatic Escaping

Django’s template engine does not trust the data being passed to it. By default, it "escapes" specific characters that are meaningful in HTML.

When Django encounters characters that could be interpreted as code, it replaces them with "HTML Entities." Here is how it transforms dangerous input into harmless text:

Character	Dangerous Meaning	Django's Escaped Version
<	Starts an HTML tag	<
>	Ends an HTML tag	>
'	Attribute delimiter	'
"	Attribute delimiter	"
&	Starts an entity	&

Example in Action:

If your view passes a variable comment = "<script>" to the template:

- **In the Browser Source:** It will appear as <script>.
- **On the Screen:** The user simply sees the literal text <script> instead of a running script.

3. When to Turn it Off (The |safe Filter)

There are rare cases where you *want* to render raw HTML (for example, if you are using a trusted Rich Text Editor). Django allows you to explicitly tell the engine, "I trust this data" using the safe filter:

```
<h1>Hello {{ user_input }}</h1>

<h1>Hello {{ user_input|safe }}</h1>
```

3.9 Create Templates Folder

Inside students/ create:

```
students/
└── templates/
    └── students/
        └── home.html
```

- ✅ So the full path is:
  students/templates/students/home.html

3.10 Create home.html

- Inside home.html, write:

```
<!DOCTYPE html>
<html>
<head>
   <title>School Portal</title>
</head>
<body>
<h1>Welcome to the School Portal</h1>
<p>This is the students app home page.</p>
</body>
</html>
```

3.11 Update View to Render Template

Open students/views.py and update it:

```
from django.shortcuts import render

def home(request):
    return render(request, 'students/home.html')
```

- Now restart server and open:
  - http://127.0.0.1:8000/students/
- ✅ You'll see your HTML page.

Mini Project Feature: Dynamic URL (Student Details)

Now we create a page like:
/students/5/ → shows student #5
/students/10/ → shows student #10

3.12 Create Student Detail View

Open students/views.py and add:

```
from django.shortcuts import render
from django.http import HttpResponse

def home(request):
    return render(request, 'students/home.html')

def student_detail(request, student_id):
    return HttpResponse(f"Student Profile Page - ID: {student_id}")
```

3.13 Add Dynamic URL Pattern

Open students/urls.py and update:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='students_home'),
    path('<int:student_id>/', views.student_detail, name='student_detail'),
]
```

3.14 Test Dynamic URLs

- Run server: python manage.py runserver
- Open:
  - http://127.0.0.1:8000/students/3/
  - http://127.0.0.1:8000/students/20/
- You will see:
- Student Profile Page - ID: 3
- ✅ That proves URL parameters work.

Section 3 Summary

By the end of Section 3, you can:

- ✅ Create an app using startapp
- ✅ Add it to INSTALLED_APPS
- ✅ Create views in views.py
- ✅ Create URL routes using urls.py
- ✅ Connect app URLs using include()
- ✅ Use templates and render HTML pages
- ✅ Build dynamic URLs with <int:id>